

Week- 1

Backend Engineering

Pawan Dhanjani
Launchpad

Object Oriented Programming in Java

Agenda

- Classes And Objects
- Packages in Java
- OOPS pillars
- Encapsulation
- QA
- Inheritance
- Abstraction
- Polymorphism
- Live Coding

Introduction to Classes and Objects

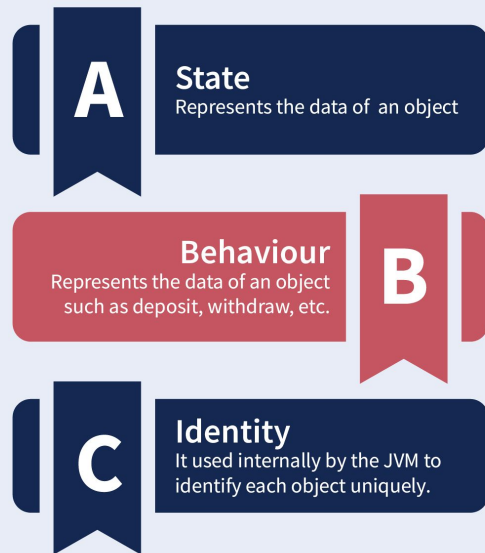
► Class:

- Blueprint for creating objects
- Defines properties (fields) and behaviors (methods)

► Object:

- Instance of a class
- Holds state (fields) and behavior (methods)

Characteristics of Object



Understanding Classes And Objects

Java Class & Objects

Class

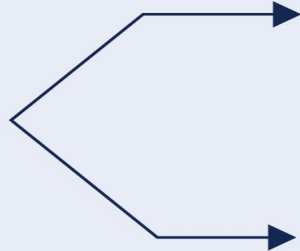
Person

Data
Members

unique_id
name
age
city
gender

Methods

eat()
study()
sleep()
play()



name- Parsoon
age- 25
city- Delhi
gender- male



name- Purva
age- 28
city- Goa
gender- female

Constructors

- A constructor is a special method used to initialize objects in Java. It is called when an object of a class is created. Constructors are essential because they set the initial state of an object by assigning values to its attributes. Unlike regular methods, constructors do not have a return type, not even void.

Types of constructors:

- Default constructor
- No argument constructor
- Parameterized constructor
- All Argument constructor
- Copy constructor

Class Definition

```
J public class Car { Untitled-1 ●
```

Class Name

```
1 public class Car {
```

Instance variables

```
2 // Properties of the Car
```

```
3 private String color;
```

```
4 private String model;
```

Local variables

```
6 public Car(String color, String model) {
```

Constructor

```
7     this.color = color;
```

```
8     this.model = model;
```

Class body

```
9 }
```

```
11 // Method to display car details
```

Instance method

```
12 public void displayDetails() {
```

```
13     System.out.println("Car model: " + model + ", Color: " + color);
```

```
14 }
```

```
15 }
```

```
16
```

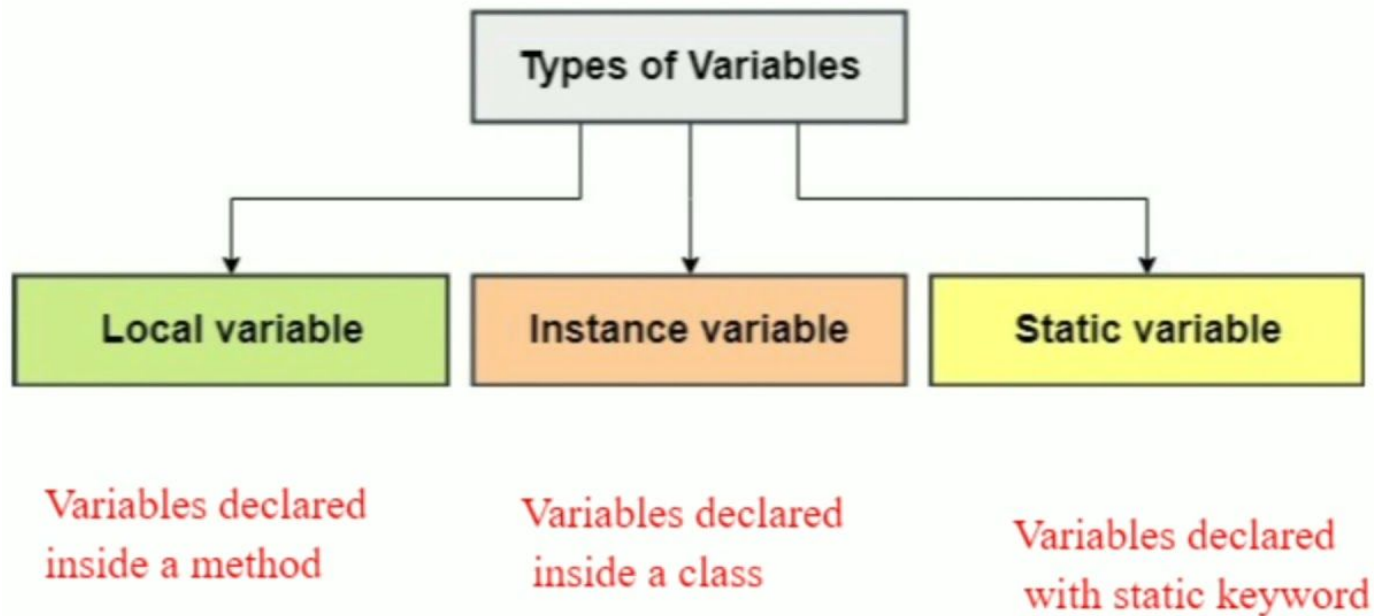
Object creation

```
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car("Red", "Tesla");  
        myCar.displayDetails();  
    }  
}
```

*Object creation by
invoking constructor*

*Invoking instance method on
myCar object*

Variable Scope



Local Variables

- ▶ **Definition:** Variables declared **within a method**, constructor, or block.
- ▶ **Scope:** Accessible **only within the method/block** where they are declared.
- ▶ **Lifetime:** Exist **only during the execution** of the method/block. They are destroyed once the method exits.
- ▶ **Default Value:** No default value (must be initialized before use).

Instance Variables

- ▶ **Definition:** Variables declared inside a class but **outside any method**, constructor, or block.
- ▶ **Scope:** Each instance (object) of the class has its own copy. Accessible by all methods of the class.
- ▶ **Lifetime:** Exist as long as the object (instance) exists. When the object is destroyed (garbage collected), instance variables are destroyed.
- ▶ **Default Value:** Have default values (e.g., 0 for numbers, null for objects).

Class variables

- **Definition:** Declared with the keyword **static** inside a class but outside any method/block.
- **Scope:** Shared among all instances of the class (single copy per class).
- **Lifetime:** Exists for the entire lifetime of the class (till the program ends or class is unloaded).
- **Default Value:** Have default values similar to instance variables.

Packages in java

- ▶ In Java, a package is a container for a collection of classes, sub-packages, and interfaces. Packages are used for the following purposes:
- ▶ Keeping name problems at bay. For example, in two packages, `college.staff.cse.Employee` and `college.staff.ee.Employee`, there could be two classes named `Employee`.
 - Making it easier to search for, locate, and use classes, interfaces, enumerations, and annotations

Packages in java

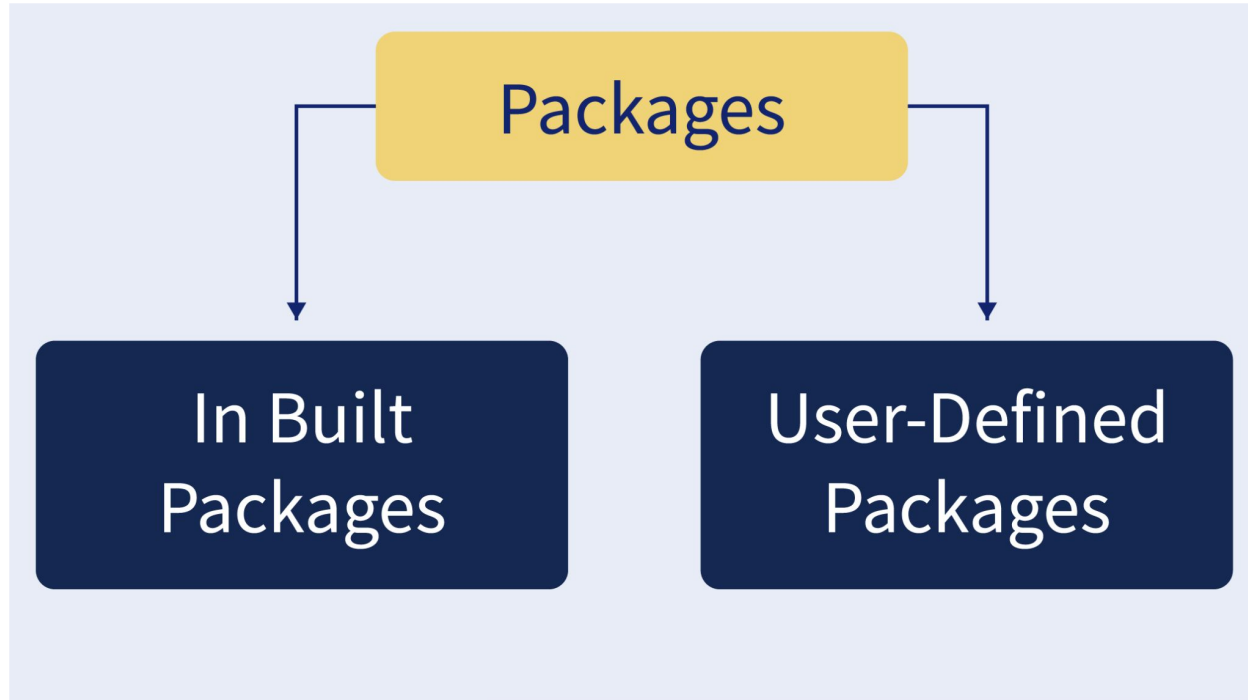
- ▶ Controlling access: package level access control is available in both protected and default modes. Classes in the same package and its subclasses can access a protected member. Only classes in the same package have access to a default member (which has no access specifier).
- ▶ Packages help in data encapsulation (or data-hiding).
- ▶ Subpackages - Subpackages are packages that are contained within another package. These are not automatically imported; they must be imported manually. Members of a subpackage also have no access privileges, therefore they are treated differently by protected and default access specifiers.

Packages in java

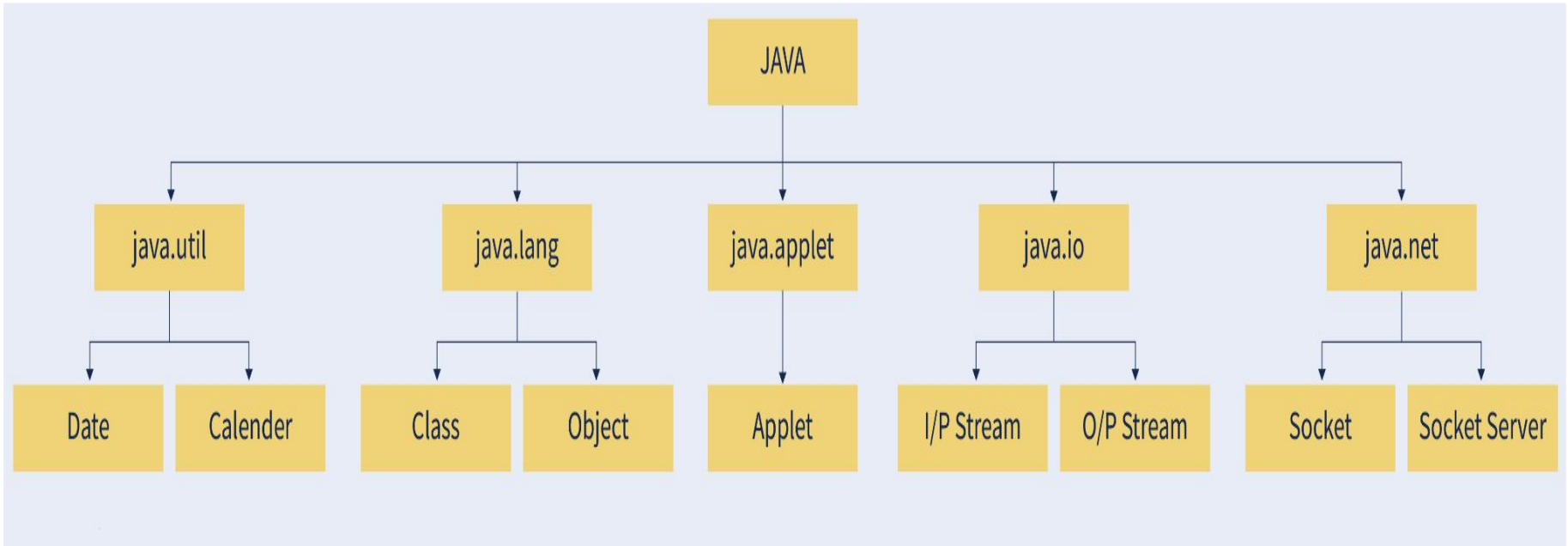
Package Benefits:

- ▶ **Namespace Management:** Avoids naming conflicts.
- ▶ **Access Protection:** Controls access to classes and interfaces.
- ▶ **Code Organization:** Organizes related classes and interfaces.

Packages in java



In Built Packages In Java

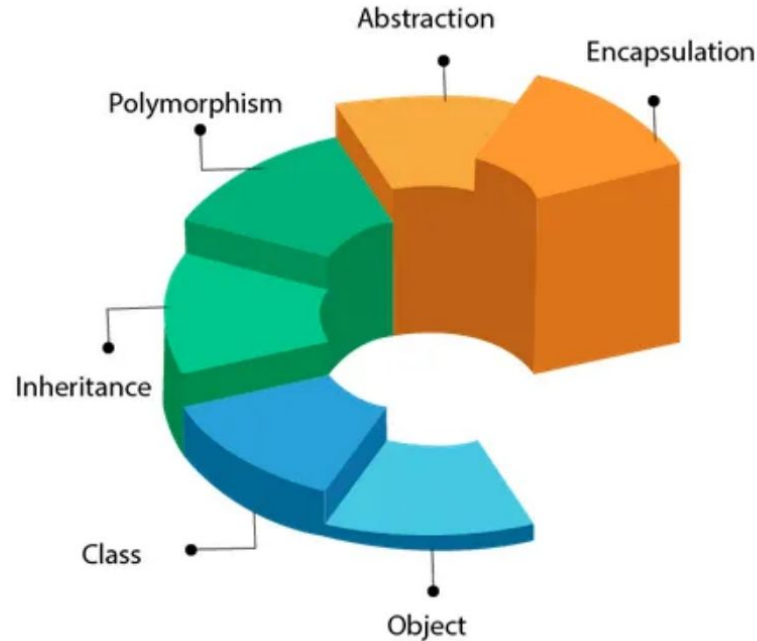


Object Oriented Programming

- ▶ A programming paradigm based on the concept of "objects", which can contain data and methods.
- ▶ Emphasizes modularity, reusability, and flexibility.
- ▶ Objects are instances of classes, which define the properties and behaviors.
- ▶ Pillars of Object Oriented Programming
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Abstraction

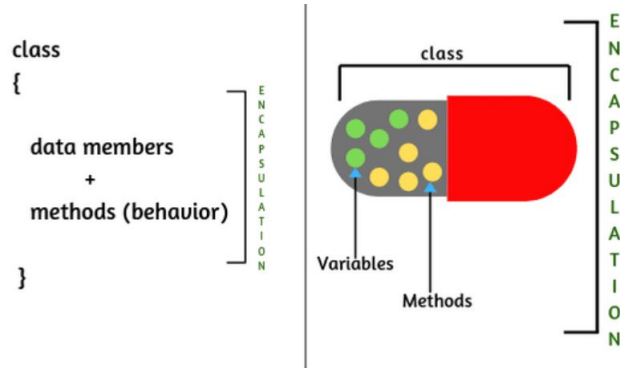
Object Oriented Programming

OOPs (Object-Oriented Programming System):



Encapsulation

- ▶ Bundling of data (variables) and methods (functions) that operate on the data into a single unit called a class.
- ▶ Protects the internal state of an object from unintended modifications.
- ▶ Access to the data is restricted through public methods (getters and setters).
- ▶ Ensures data integrity and security.
- ▶ Increases code maintainability and flexibility.



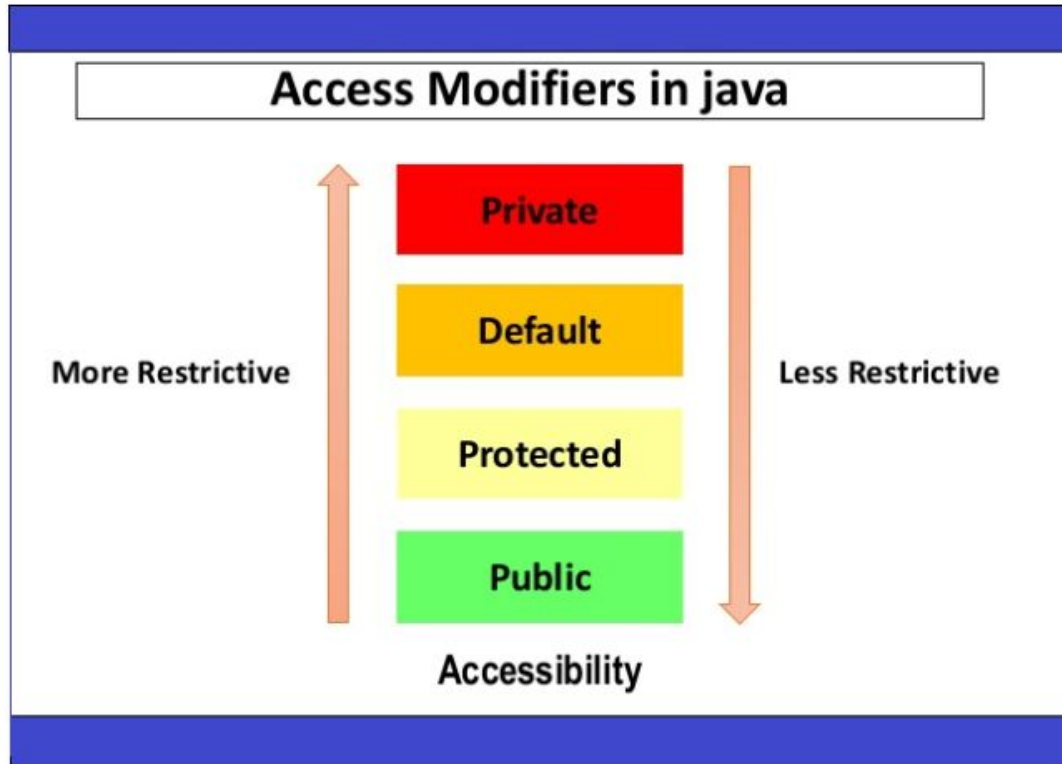
Access Modifiers

- ▶ Access modifiers in Java determine the visibility and accessibility of classes, methods, and variables. They help implement encapsulation and control the exposure of class members to other parts of the program. Java provides four access modifiers: **public**, **protected**, **default** (package-private), and **private**.
- ▶ The **public** access modifier makes the class, method, or variable accessible from any other class.
- ▶ The **protected** access modifier makes the method or variable accessible within the same package and by subclasses in different packages.

Access Modifiers

- ▶ When no access modifier is specified, Java uses the default access level (package-private), making the method or variable accessible only within the same package.
- ▶ The **private** access modifier makes the method or variable accessible only within the class it is declared in.

Access Modifiers



Access Modifiers

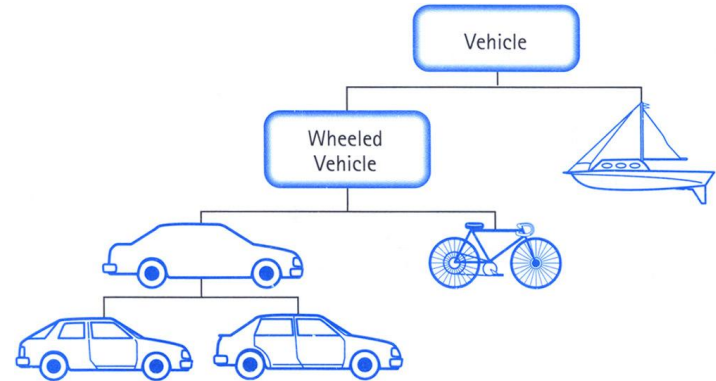
- **Scope of Access Modifiers:**
- **Public:** Accessible from any other class.
- **Protected:** Accessible within the same package and by subclasses in other packages.
- **Default (Package-private):** Accessible only within the same package.
- **Private:** Accessible only within the class it is declared in.

Visibility of access Modifiers

Visibility	Private	Default	Protected	Public
Same Class	YES	YES	YES	YES
Same Package non-Subclass	NO	YES	YES	YES
Same Package Subclass	NO	YES	YES	YES
Different Package Non-Subclass	NO	NO	NO	YES
Different Package Subclass	NO	NO	YES	YES

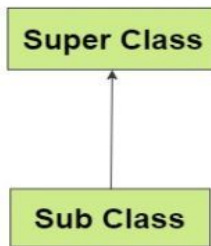
Inheritance

- Mechanism where a new class (derived class) inherits properties and behaviors from an existing class (base class).
- Promotes code reuse and the creation of hierarchical relationships between classes.
- Allows the derived class to override or extend the functionalities of the base class.
- Supports polymorphic behavior.
- Reduces code redundancy and enhances code organization

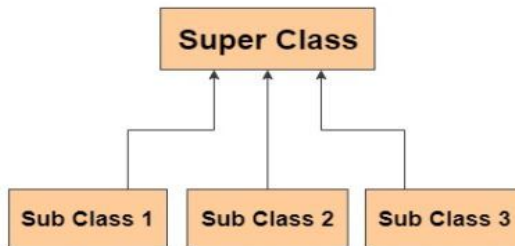


Types Of Inheritance

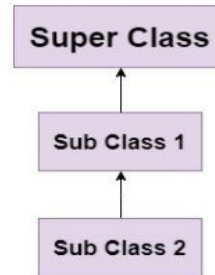
Single Inheritance



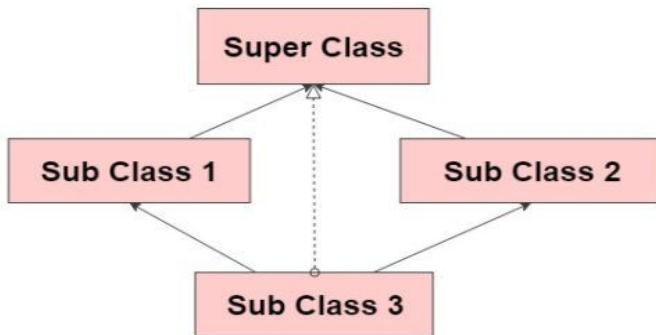
Hierarchial Inheritance



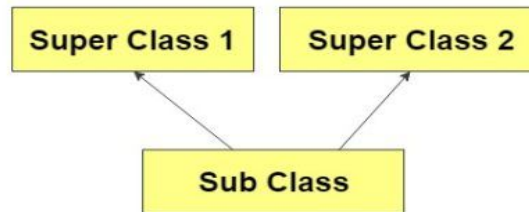
MultiLevel Inheritance



Hybrid Inheritance



Multiple Inheritance



Single Inheritance

```
1  class Car {
2      String brand;
3
4      void displayBrand() {
5          System.out.println("Brand: " + brand);
6      }
7  }
8
9  // Subclass
10 class ElectricCar extends Car {
11     int batteryLife;
12
13     void displayBatteryLife() {
14         System.out.println("Battery Life: " + batteryLife + " hours");
15     }
16 }
17
```

Multilevel Inheritance

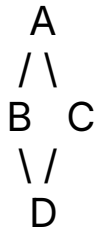
```
1  // Superclass
2  class Car {
3      String brand;
4
5      void displayBrand() {
6          System.out.println("Brand: " + brand);
7      }
8  }
9
10 // Intermediate Subclass
11 class ElectricCar extends Car {
12     int batteryLife;
13
14     void displayBatteryLife() {
15         System.out.println("Battery Life: " + batteryLife + " hours");
16     }
17 }
18
19 // Final Subclass
20 class AutonomousElectricCar extends ElectricCar {
21     boolean autonomousMode;
22
23     void displayAutonomousMode() {
24         System.out.println("Autonomous Mode: " + (autonomousMode ? "Enabled" : "Disabled"));
25     }
26 }
```

Hybrid Inheritance

```
1  class Vehicle {
2      String brand;
3
4      void displayBrand() {
5          System.out.println("Brand: " + brand);
6      }
7  }
8
9  // Intermediate class for Electric Vehicles
10 class ElectricVehicle extends Vehicle {
11     int batteryLife;
12
13     void displayBatteryLife() {
14         System.out.println("Battery Life: " + batteryLife + " hours");
15     }
16
17     void chargeBattery() {
18         System.out.println("Battery is charging...");
19     }
20 }
21
22 // Intermediate class for Autonomous Vehicles
23 class AutonomousVehicle extends Vehicle {
24     boolean autonomousMode;
25
26     void enableAutonomousMode() {
27         autonomousMode = true;
28         System.out.println("Autonomous mode enabled.");
29     }
30
31     void displayAutonomousMode() {
32         System.out.println("Autonomous Mode: " + (autonomousMode ? "Enabled" : "Disabled"));
33     }
34 }
35
36 // Hybrid class combining Electric and Autonomous features
37 class HybridCar extends ElectricVehicle & AutonomousVehicle {
38     boolean autonomousMode;
39
40     void enableAutonomousMode() {
41         autonomousMode = true;
42         System.out.println("Autonomous mode enabled.");
43     }
44
45     void displayDetails() {
46         displayBrand();
47         displayBatteryLife();
48         displayAutonomousMode();
49     }
50
51     void displayAutonomousMode() {
52         System.out.println("Autonomous Mode: " + (autonomousMode ? "Enabled" : "Disabled"));
53     }
54 }
```

Diamond Problem – Multiple Inheritance

- ▶ The diamond problem occurs in multiple inheritance when two classes inherit from a common superclass and a third class inherits from these two classes, leading to ambiguity. Java avoids this problem by not supporting multiple inheritance with classes.
- ▶ Java does not allow a class to inherit from more than one superclass. This restriction prevents the diamond problem, which can occur when there is ambiguity in the inheritance hierarchy. If multiple inheritance were allowed, a class could inherit the same method from multiple superclasses, causing confusion about which method to execute.



This and super keyword

- ▶ The **this** keyword in Java is a reference to the current object. It is used to eliminate confusion between class attributes and parameters with the same name, to call other constructors in the same class, and to pass the current object as a parameter to other methods.
- ▶ The **super** keyword in Java is a reference to the parent class object. It is used to call the superclass's methods and constructors, and to access the superclass variables.

This and super keyword

► Uses of *this* keyword

► Referencing Instance Variables

- When parameter names are the same as instance variable names, **this** is used to differentiate between the instance variable and the parameter.

► Calling Another Constructor

- **this()** can be used to call another constructor in the same class. This is known as constructor chaining within the same class.

► Passing Current Object as a Parameter

- **this** can be used to pass the current object as a parameter to another method.

This and super keyword

► Uses of *super* keyword

► Calling Superclass Methods

- **super** is used to call a method from the superclass that has been overridden in the subclass.

► Accessing Superclass Variables

- **super** is used to access superclass variables when they are shadowed by subclass variables.

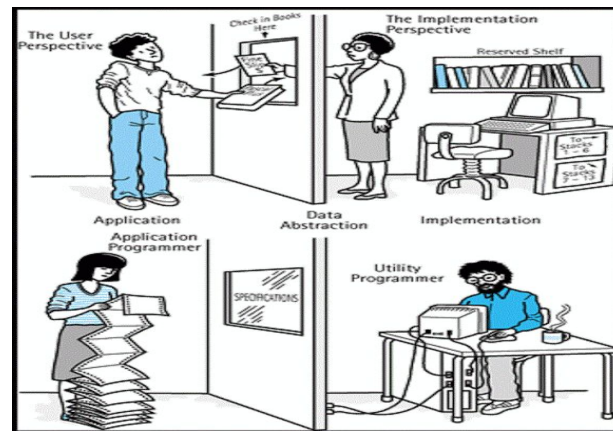
► Calling Superclass Constructor

- **super()** is used to call the superclass's constructor. This is useful for initializing the superclass part of the subclass object.

It's time for the QUIZ

Abstraction

- ▶ Hiding the complex implementation details and showing only the essential features of the object.
- ▶ Focuses on what an object does rather than how it does it.
- ▶ Abstract classes and interfaces are used to achieve abstraction.
- ▶ Simplifies complex systems by breaking them into smaller, manageable units.
- ▶ Enhances code readability and reduces complexity.



Abstraction using abstract class

► An abstract class is a class that cannot be instantiated on its own and is meant to be subclassed. It can have abstract methods (methods without a body) and concrete methods (methods with a body). Abstract classes are used to define a common interface for subclasses.

► **Characteristics:**

- Can have both abstract and concrete methods.
- Can have member variables.
- Can have constructors.
- Can implement methods from an interface.
- Cannot be instantiated.

Abstraction using Interface

► An interface in Java is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types. Interfaces cannot contain instance fields or constructors. All methods in an interface are abstract by default (except default and static methods).

► **Characteristics:**

- Can only have abstract methods (until Java 7).
- From Java 8, can have default and static methods.
- From Java 9, can have private methods.
- Cannot have constructors.
- Cannot have instance variables, only static and final variables.
- A class can implement multiple interfaces.

Abstract class vs interface

Interface

I only know method names that I will require for my job to be done.
You have to provide body for those methods.

Interface

Sure, I will definitely provide body to all your methods but in my way.

Implementer

contract



Abstract class

abstract class

Some methods I know.

Some methods I don't know and I will depend upon you to provide.

Implementer 2



Some methods I know.

Some methods I don't know and I will depend upon you to provide.

Implementer 1



Interface versus Concrete class

Feature	Interface	Concrete Class
Instantiation	Cannot be instantiated	Can be instantiated
Methods	Can have abstract, default, and static methods	Can have concrete methods, static methods
Variables	Can only have static and final variables	Can have instance variables

Abstract class versus Concrete class

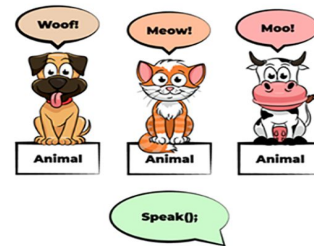
Feature	Abstract Class	Concrete Class
Instantiation	Cannot be instantiated	Can be instantiated
Methods	Can have abstract, default, and static methods	Can have concrete and static methods
Variables	Can have instance variables , static and final variables	Can have instance variables , static and final variables

Abstract class versus Interface

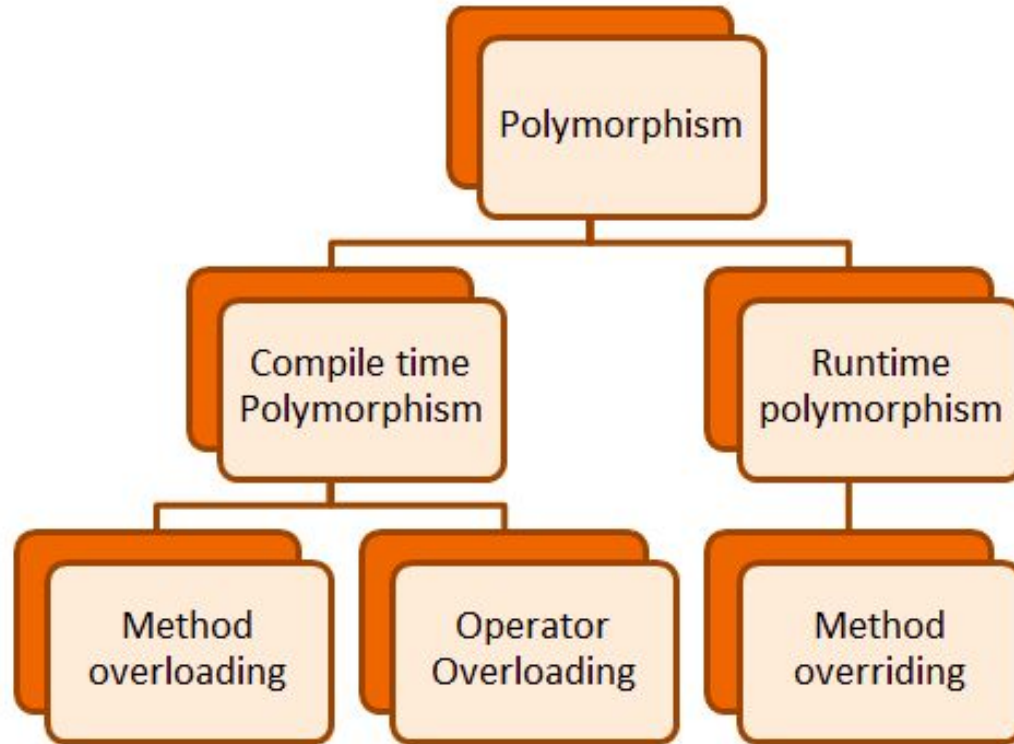
Feature	Abstract Class	Interface
Methods	Can have abstract and concrete methods	Can have abstract, default, and static methods (since Java 8)
Variables	Can have instance variables	Can only have static and final variables
Multiple Inheritance	Does not support multiple inheritance	Supports multiple inheritance
Constructors	Can have constructors	Cannot have constructors
Implementation	Can provide method implementation	Cannot provide method implementation (except default and static methods)

Polymorphism

- ▶ Ability of a variable, function, or object to take multiple forms.
- ▶ Two types: Compile-time polymorphism (method overloading) and Runtime polymorphism (method overriding).
- ▶ Allows objects of different classes to be treated as objects of a common superclass.
- ▶ Facilitates the design of extensible and maintainable systems.
- ▶ Enhances flexibility and scalability of code.



Types of polymorphism



Understanding Polymorphism

Feature	Compile-Time Polymorphism	Runtime Polymorphism
Achieved By	Method Overloading	Method Overriding
Resolution Time	Compile Time	Runtime
Binding	Early Binding	Late Binding
Method Signature	Same method name but different parameters	Same method name and parameters
Inheritance Required	No	Yes
Performance	Faster (resolved at compile time)	Slower (resolved at runtime)
Flexibility	Less flexible, determined at compile time	More flexible, determined at runtime

Method and constructor Overloading

- ▶ Method overloading is a feature that allows a class to have more than one method having the same name, if their parameter lists are different. It is related to compile-time (or static) polymorphism.
- ▶ Constructor overloading in Java is a technique that allows a class to have more than one constructor, each with a different parameter list. These constructors perform different tasks based on the different sets of parameters they accept.

Method overload resolution

- ▶ Method overload resolution is the process by which the Java compiler determines which method to call when multiple methods with the same name but different parameters exist in a class. The decision is based on the method signature, which includes the number, type, and order of parameters.

- ▶ **Method Overload Resolution Process:**

1. **Exact Match:** The compiler first looks for an exact match of the method signature. If it finds one, it calls that method.
2. **Type Conversion:** If no exact match is found, the compiler tries to apply type conversion (automatic promotion) to the arguments to find a matching method.
3. **Ambiguity:** If multiple methods are applicable after type conversion, the compiler may throw an error due to ambiguity.

Automatic Promotion

- Automatic promotion (also known as type promotion) occurs when Java automatically converts a smaller data type to a larger data type during method overloading. This happens if there is no exact match for the method signature and the smaller data type can be promoted to a larger one.

- **Promotion Rules:**

1. **byte → short → int → long → float → double**
2. **char → int → long → float → double**

Constructor Overloading

```
1 class Car {
2     String brand;
3     int batteryLife;
4     boolean autonomousMode;
5
6     // Default constructor
7     Car() {
8         this.brand = "Unknown";
9         this.batteryLife = 0;
10        this.autonomousMode = false;
11    }
12
13    // Constructor with one parameter
14    Car(String brand) {
15        this.brand = brand;
16        this.batteryLife = 0;
17        this.autonomousMode = false;
18    }
19
20    // Constructor with two parameters
21    Car(String brand, int batteryLife) {
22        this.brand = brand;
23        this.batteryLife = batteryLife;
24        this.autonomousMode = false;
25    }
26
27    // Constructor with three parameters
28    Car(String brand, int batteryLife, boolean autonomousMode) {
29        this.brand = brand;
30        this.batteryLife = batteryLife;
31        this.autonomousMode = autonomousMode;
32    }
33
34    void displayDetails() {
35        System.out.println("Brand: " + brand);
36        System.out.println("Battery Life: " + batteryLife + " hours");
37        System.out.println("Autonomous Mode: " + (autonomousMode ? "Enabled" : "Disabled"));
38    }
39 }
```

Method Overloading

```
1  class Car {  
2      String brand;  
3      int batteryLife;  
4      boolean autonomousMode;  
5  
6      Car(String brand, int batteryLife, boolean autonomousMode) {  
7          this.brand = brand;  
8          this.batteryLife = batteryLife;  
9          this.autonomousMode = autonomousMode;  
10     }  
11  
12     // Overloaded method with three parameters  
13     void displayDetails(String owner, String registrationNumber, String color) {  
14  
15     }  
16  
17     // Overloaded methods to display car details  
18     void displayDetails() {  
19  
20     }  
21  
22     // Overloaded method with one parameter  
23     void displayDetails(String owner) {  
24  
25     }  
26  
27     // Overloaded method with two parameters  
28     void displayDetails(String owner, String registrationNumber) {  
29  
30     }  
31 }  
32
```

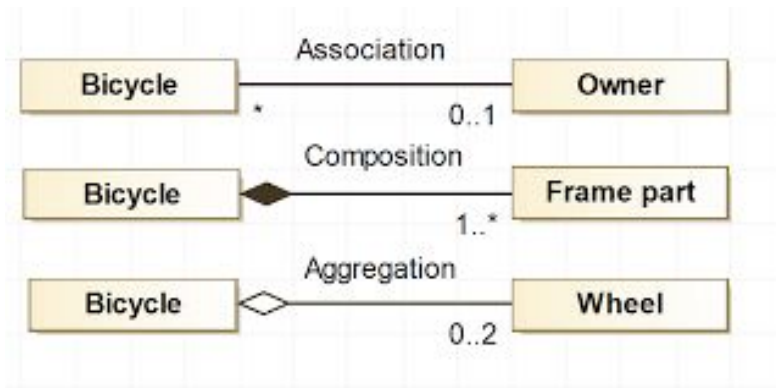
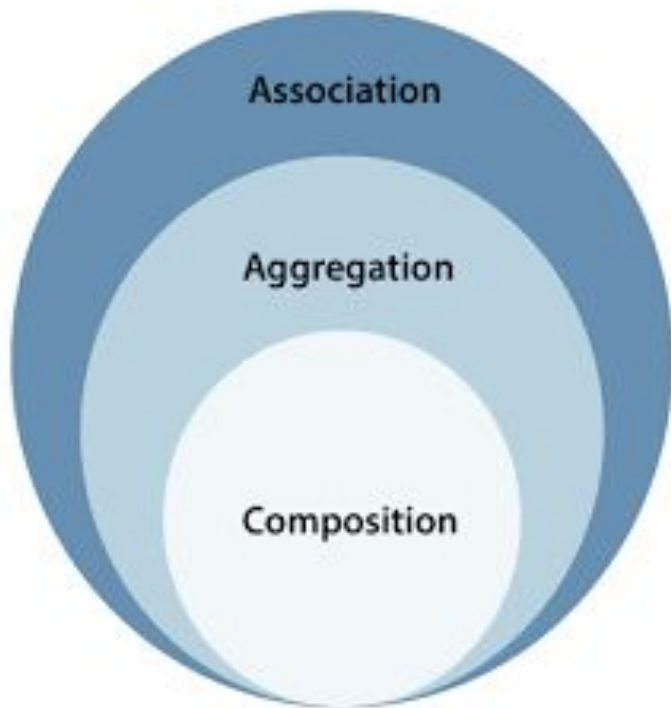
Method overriding

- ▶ Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass. The overridden method in the subclass should have the same signature (name, return type, and parameters) as in the superclass.

- ▶ **Dynamic Method Dispatch:**

Java uses a technique called dynamic method dispatch to resolve calls to overridden methods at runtime. When an overridden method is called through a superclass reference, Java determines which version of the method (superclass or subclass) to execute based on the actual object that the reference points to.

Relationships in OOPS



Association

- **An association** can be considered a generic term to indicate the relationship between two independent classes; the relationship may be **one-to-one**, **one-to-many**, or **many-to-many**, but it need not indicate ownership.
- A general relationship between two classes that allows one object to use the functionality of another.

Key Points:

- Represents a "uses-a" relationship.
- Objects are independent of each other.
- No ownership is implied.

A **Car** and a **Driver** are associated because a car is driven by a driver, but neither owns the other.

Aggregation

- ▶ **Aggregation** is a specific form of association in which one class, the whole, contains a collection of other classes, the parts; here, however, the lifecycle of the parts does not depend upon the whole. For example, a library and books show aggregation, since books may exist somewhere apart from the library.
- ▶ Aggregation represents a "**has-a**" **relationship** where the parent object owns or references the child object, but the child can exist independently of the parent.

Key Points:

- It shows a **weak ownership** relationship.
- The lifetime of the parent and child objects are **independent**.
- If the parent is removed, the child continues to exist.

Aggregation

► Examples:

- A **Library** contains **Books**, but a **Book** can exist without the **Library**.
- A **Team** has a **Coach**, but the **Coach** is not dependent on the **Team**.

Composition

- **composition** is a stronger form of aggregation that means ownership and lifecycle dependence; if the whole gets destroyed, then the parts no longer exist. For composition, a good example would be a house and its different rooms; a room cannot exist without a house.

A stronger form of aggregation where the child objects are **owned** and **lifetime-bound** to the parent object.

Key Points:

- Represents a "has-a" relationship with **strong ownership**.
- The child cannot exist without the parent.
- When the parent is destroyed, the child is also destroyed.

Composition

► Example:

- A **House** has **Rooms**, and if the **House** is demolished, the **Rooms** no longer exist.

Difference b/w Association, Aggregation and Composition

Aspect	Association	Aggregation	Composition
Relationship Type	"Uses-a"	"Has-a" (weak ownership)	"Has-a" (strong ownership)
Ownership	No ownership	Weak ownership	Strong ownership
Dependency	Objects are independent	Parent and child are loosely coupled	Child depends on the parent
Lifetime	Independent	Independent	Dependent
Example	Driver ↔ Car	Library → Books	House → Rooms

**Time for some
ACTION**

Airtribe Learner Manager

Associated session – session #2 (Java Basics – OOPS)

You need to develop a Learner Management System (LMS) for Airtribe in Java, emphasizing Object-Oriented Programming (OOP) principles such as classes, encapsulation, abstraction, and inheritance. Your program should demonstrate understanding of Java OOPS covered in this week's session.

LMS should cater to the following types of learners

- Node JS Learners
- Java Learners

Airtribe is also exploring a model to have **Offline** and **Online** courses, with offline course being taught physically and online courses being taught over online calls.

Learner Management System

You should supporting following features

- 1) Creation of the new batches for node JS + Java courses (Offline + Online)
- 2) Creation and enrollment of new learners (Node Learner + Java Learner)
- 3) Creation of the cohorts, assignment of the learners to the cohorts
- 4) Xp's for the learners, along with the methods to support average computation of the xp's
- 5) Displaying the details of the course, cohort, learners, instructors etc

Learner Management System

1. **Course Management:**
 - Create a course with attributes specifying:
 - Type (online or offline).
 - Course specialization (Node.js or Java).
2. **Cohort Management:**
 - Create a cohort with:
 - Learners enrolled in the cohort.
 - An instructor assigned to the cohort.
3. **Instructor Management:**
 - Create an instructor with a unique identifier and name.
4. **Learner Management:**
 - Create a Node.js learner or Java learner and assign them to the corresponding cohort.
 - Each learner should have:
 - A unique identifier and name.
 - Experience points (XP) specific to their course.
 - Ability to calculate and display their XP.
5. **Functionality:**
 - Calculate the average XP of all learners in a cohort.
6. **Display Details:**
 - Display details of courses, cohorts, instructors, and learners.

Got Questions?

Thank you